

A25

PROJET AKROPOLIS

- LO21 -

Rapport n°1

ANCELY Louise
BLIARD Samuel
GENGEMBRE Gabriel
LAVALLEE Antoine
PIERRE Romain

Introduction.....	1
1. Rappel des règles.....	2
2. Concepts identifiés.....	2
2.1 Entités principales.....	2
2.2 Types de quartiers.....	2
2.3 Autres éléments.....	3
3. Contraintes imposées et premières réflexions.....	3
4. Modèle Conceptuel.....	4
5. Planning prévisionnel.....	4

Introduction

Ce premier rapport a pour objectif de définir les concepts fondamentaux du jeu Akropolis. Il présente un rappel des règles principales du jeu, l'identification des entités qui le composent ainsi qu'un premier modèle conceptuel qui servira de base aux étapes suivantes de conception. Ce document joue donc le rôle d'un cahier des charges initial. Il permet d'avoir une vision claire et globale sur le jeu et prépare la planification du projet en vue de l'élaboration progressive de plusieurs prototypes d'architectures logicielles.

1. Rappel des règles

Le but du jeu Akropolis est de placer des tuiles afin de construire une cité. Chaque tuile rapporte des points au joueur en fonction de certaines contraintes de placement.

- Chaque tuile cité est composée de 3 hexagones construction.
- Les hexagones peuvent représenter :
 - un quartier (habitation, marché, caserne, temple, jardin),
 - une place (multiplicateur de points),
 - une carrière (source de pierres).
- Les quartiers rapportent des points selon leur type et leur positionnement.
- Les places ajoutent des multiplicateurs de score (étoiles).
- Les carrières ne rapportent pas directement de points, mais fournissent des pierres, qui permettent de faciliter le choix des tuiles et ajoutent des points en fin de partie.
- Les joueurs posent leurs tuiles de manière adjacente ou en superposition.
- La partie se termine à l'épuisement des tuiles. Le vainqueur est celui qui a le plus de points.

2. Concepts identifiés

2.1 Entités principales

- Partie : représente une session de jeu, avec son déroulement et ses paramètres (joueurs, variantes, mode solo, etc.).
- Joueur : humain ou virtuel (*Illustre Constructeur*), possède une cité, des pierres et un score.
- Cité : l'ensemble de tuiles construites par un joueur.
- Tuile cité : unité de jeu composée de 3 hexagones.
- Hexagone construction : élément de base, qui peut être un quartier, une place ou une carrière.
- Tableau des scores : Stocke les différentes variables du joueurs (points des quartiers, des places, points totaux, nombre de pierres...)
- Pioche de tuiles : espace de stockage de toutes les tuiles restantes à placer.

2.2 Types de quartiers

- Habitation (bleu) : seules celles du plus grand groupe rapportent des points.
- Marché (jaune) : ne doit pas être adjacent à un autre marché.
- Caserne (rouge) : doit être placée en périphérie de la cité.
- Temple (violet) : doit être entièrement entouré.
- Jardin (vert) : rapporte toujours des points, sans contrainte.

2.3 Autres éléments

- Place : multiplicateur de points associé à un type de quartier, cumulable.
- Carrière : produit des pierres lorsqu'elle est recouverte par une nouvelle tuile.
- Pierre : ressource permettant de modifier les choix de tuiles et rapportant des points en fin de partie.

3. Contraintes imposées et premières réflexions

- Contraintes de placement : contraintes spécifiques au positionnement (ex. temples entourés, casernes en périphérie). Il faut prévoir la rotation des tuiles et la référence de l'hexagone par rapport à la rotation. Le plateau de jeu est composé d'emplacement hexagonal et il est infini. La superposition de tuiles cités peut engendrer des restrictions de placement pour les tuiles suivantes.
- Contraintes de scoring : définissent le calcul des points de victoire en fonction des quartiers, places et pierres. Il faut calculer la position de chaque hexagone à chaque tour par rapport aux autres afin d'ajouter mais aussi d'enlever des points si certaines restrictions ne sont plus respectées. La superposition de tuiles cités permet de gagner des pierres mais aussi plus de points (niveau 2 = les points de la tuile citée sont doublés, niveau 3 = ils sont triplés).
- Contraintes sur la pioche : toujours les mêmes tuiles mais mélangées. Diminution du coût (en pierre) des tuiles à chaque tour. Quand il reste une tuile, la pioche en propose 3 autres (on a donc la possibilité de choisir entre 4 tuiles et non 3). La dernière tuile n'est pas toujours posée (si la victoire est évidente).

- Contraintes supplémentaires : faire varier le nombre de joueurs de 1 à 4 en intégrant leur nom (il faut être attentif à l'interface graphique pour la jouabilité). Si le joueur est seul, il joue alors contre "l'Illustre Architecte", l'interface est similaire mais les règles du jeu sont différentes (on simule un joueur virtuel). Les règles changent en fonction du niveau de difficulté en mode solo (3 niveaux différents). Le nombre de tuiles est variable (standard ou augmenté). Il faut aussi pouvoir jouer en mode console ou en mode graphique mais le mode console est celui à développer en priorité. Pour finir, on peut développer, en fin de projet, plusieurs variantes du jeu (règles différentes).

4. Modèle Conceptuel

Partie : id(clé), mode, variante

- possède des Joueur (1..4 - 1)
- possède une Pioche (1 - 1)

Joueur : nom(clé), NbPierres

- possède un TableauScore (1 - 1)
- possède une Cité (1 - 1)

Pioche : id(clé)

- est composée de TuileCité (0 - 11)

TableauScore : id(clé), ScoreTotal, NbEtoilesHabitation, NbHabitationMax, NbEtoilesMarché, NbMarchésSeuls, NbEtoilesCaserne, NbCasernesExte, NbEtoilesTemple, NbTemplesEntour, NbEtoilesJardin, NbJardins

Cité : id(clé)

- contient des TuileCité (0 - 11)

TuileCité : id(clé), NbBrique

- comporte des HexagoneConstruction (3 - 1)

HexagoneConstruction :

- peut comporter des Quartiers (0..3 - 0..*)
- peut comporter des Carrières (0..3 - 0..*)
- peut comporter des Places (0..3 - 0..*)

Quartier : id(clé)

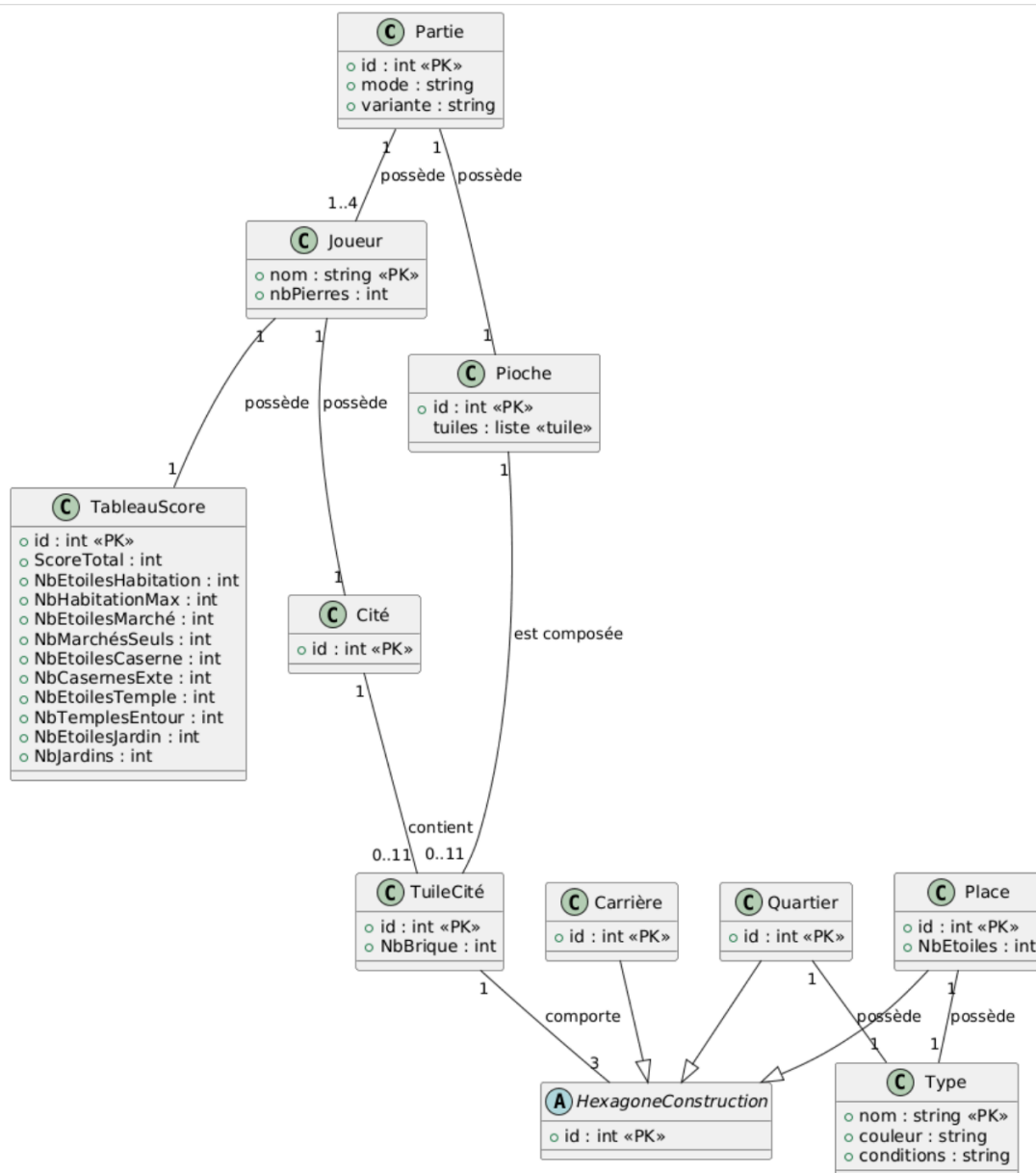
- possède un Type (1 - *)

Carrière : id(clé)

Places : id(clé), NbEtoiles

- possède un Type (1 - *)

Type : nom(clé), couleur, conditions



5. Planning prévisionnel

Tâche principales	Durée	Date limite
Architecture Logicielle / Modélisation des classes - Jouer au jeu afin d'identifier les concepts clés comme les tuiles, les hexagones, les quartiers, les joueurs et les places. - L'objectif est de créer un MCD qui sera présent dans le premier livrable.	18h	28/09/25

- Créer les structures en C++.		
Gestion des relations et contraintes (ex : max 3 quartiers, carrières ou places.) - Écrire le code qui gère les interactions entre les différents éléments du jeu. - Inclut toutes les contraintes et relations mentionnées dans ce rapport.	17h	05/10/25
Création des cartes dans la mémoire - Passer sous forme de code les différents objets de classe que nous souhaitons inclure dans le jeu.	8-10h	12/10/25
Implémentation des méthodes - Écriture du code qui permet logique qui permet de vérifier la validité des actions des joueurs et de calculer les points. - Il sera nécessaire d'implémenter les méthodes de scoring pour chaque type de quartier et de place	18-20h	02/11/25
Gestion des joueurs et du tour de jeu - Mettre en œuvre le système de tours, la gestion des joueurs - Créer le mode solo avec un Illustre Constructeur qui aura plusieurs niveaux de difficulté. - Le jeu doit se dérouler jusqu'à l'épuisement des tuiles.	15-20h	16/11/25
Interface utilisateur (menu, liste des options, etc) - Développer une interface qui donne envie aux joueurs. - Faire une interface en mode console et une autre graphique avec le framework Qt - L'interface doit permettre - Le paramétrage d'une partie, - La visualisation de l'état du jeu - La vérification de la validité des actions	45-50h	30/11/25
Tests, à vérifier : - Que toutes les fonctionnalités soient bien implémentées - Le calcul des points - La validation des actions - ...	10h	14/12/25
Rendu du projet, qui inclut : - Le code source - Une vidéo de présentation - Un rapport détaillé		21/12/25

Architecture Logicielle / Modélisation des classes	Durée	Affectation
Créer la classe Partie	2h	Louise
Créer Joueur	2h	Romain
Créer Pioche	2h	Gabriel
Créer Tableau des scores	2h	Antoine
Créer Cité	2h	Samuel
Créer TuileCite	2h	Louise
Créer HexagoneConstruction	2h	Romain
Créer les classes Quartier, Carrière, Place	2h	Gabriel
Créer la classe Type	2h	Louise

Gestion des relations et des contraintes	Durée	Affectation
Vérification de la validité d'un placement de tuile	6h	Samuel
Inclure les contraintes de placement des différents quartiers	5h	Louise
Gérer les carrières	3h	Romain
Calculer les points	3h	Gabriel

Création des cartes dans la mémoire	Durée	Affectation
Générer les tuiles de jeu	4h	Antoine
Gérer le mélange et la distribution des tuiles	4h	Samuel
Initialiser la réserve de pierres pour chacun des joueurs	1h	Louise

Implémentation des méthodes	Durée	Affectation
Implémenter le système de comptage des points en fonction des contraintes	5h	Romain
Implémenter les multiplicateurs avec les places	3h	Gabriel
Implémenter l'utilisation des pierres	4h	Antoine
Intégrer des méthodes de vérification des conditions	5h	Samuel

Gestion des joueurs et des tours de jeu	Durée	Affectation
Gérer la pioche et le choix des tuiles	5h	Romain
Gestion du tours à tours	2h	Gabriel
Gérer le mode solo (contre ordinateur)	10h	Gabriel et Louise

Interface utilisateur	Durée	Affectation
mode console (priorité) :	total 35h	Romain et Samuel
menu (afficher les options : nouvelle/charger une partie)	10h	Louise
affichage état du jeu (tuile disponible, plateau du joueur ,score)	25h	Antoine
mode graphique (Qt)	10-15h	Gabriel

Tests	Durée	Affectation
test interne au tours (tours a tours)	2h	Gabriel
test enchaînements des tours	4h	Louise
test partie complète (6-7 partie)	4h	Louise

Conclusion

Ce premier travail d'analyse a permis d'identifier les concepts essentiels du jeu Akropolis et leurs relations, nous pouvons désormais commencer à répartir les premiers éléments de conceptions entre les différents membres du groupe.

Le modèle conceptuel, bien que basique pour le moment, nous permettra de préparer l'architecture logicielle orientée objet mais aussi d'anticiper l'évolution du projet (ajout de nouvelles tuiles, variantes, mode solo).

Le prochain rapport (semaine du 4 novembre) proposera architecture logicielle bien plus détaillée, en intégrant des modules orientés vers l'implémentation.